From Characters to Transformers: lecture outline

Status: working source of truth for the Typst mega-deck
Format: 3 × 80 minutes, one continuous deck
Target: 164 conceptual frames; progressive builds may create more presentation pages
Teaching reference: Nipun's next-character prediction notes

Frame indices in this document are planning references only. They are not printed in the slide titles.

The tables below retain the original 154-frame planning IDs from the imported conversation. Ten extra slides work through intermediate steps or show a short code example. The original planning IDs stay fixed so earlier comments remain easy to find:

Actual frame	Inserted after	Added slide	What students do
30	Compute the loss	The loss in PyTorch	Compare the hand calculation with a short loss cell.
63	First try: average the token vectors	Average one tiny example	Calculate a mean before discussing what it loses.
75	Value: what should this source send?	Make a query, key, and value	Follow one token state through its three projections.
78	Write the three steps in one line	One query in PyTorch	Match the attention calculation to four tensor operations.
81	Pick tiny vectors we can check by hand	What does the dot product notice?	Keep the query fixed. Compare perpendicular and same-direction unit vectors, then calculate their dot products.
104	Mask future scores before softmax	The causal mask in PyTorch	Match the triangular diagram to the masking code.
134	Write the block as two updates	The block in five lines of PyTorch-style code	Match each line to a path in the block diagram.
137	The next block uses the updated token states	Information can pass through an intermediate token	Follow information through two blocks.
145	The next-token objective stays the same	Shift the targets and compute cross-entropy	Align the shifted targets and flatten the rows for the loss.
150	Generation, one step later	The whole generation loop	Read the six-line sampling loop.

What students will learn

Students will build a name generator, identify what limits its context window, and work through attention and a complete Transformer. Both models learn to:

$$\boxed{\text{predict the next token from previous tokens}}$$

What changes is the architecture used to represent the context:

$$\texttt{aabid} \to \text{fixed-window MLP} \to \text{adaptive retrieval} \to \text{self-attention} \to \text{multi-head attention} \to \text{Transformer}\ .$$

Follow one small example. Write out its training rows, look up its vectors, check the shapes, and calculate an output. Ask students for the next step before revealing it. After introducing a general equation, return to the example and point to what each symbol means.

Worked example carried throughout

With context length three and one end-of-sequence prediction, `aabid` creates six examples:

$$---\to a,\quad --a\to a,\quad -aa\to b,\quad aab\to i,\quad abi\to d,\quad bid\to -.$$

Style and notation

Teaching style

Use the step-by-step teaching method in the original 23-page handwritten next-token lecture. Styling comes directly from `common/metropolis.typ`, the theme used by the earlier course decks. Typst handles the fonts, spacing, dark-teal title bar, orange accent, and bottom slide number; there is no separate CSS file or local copy of the theme.

- a plain light background, dark text, and space around the worked example;
- one held drawing is redrawn in the same location while one new annotation, row, arrow, number, or highlight appears per build;
- headings are short questions, concrete statements, or operation names;
- simple tables, axes, arrows, matrices, and network sketches are the main visual vocabulary;
- consistent colors carry token identity across lookup, concatenation, and the network; red marks losses and forbidden attention;
- boxes exist only when the idea itself is a box, token, table, or model layer; avoid generic dashboard cards and dense UI grids;
- small exact numbers appear early; compact tensor notation arrives only after students can point to the corresponding marks in the held drawing;
- ask a question, draw the example, add one operation, calculate its result, and explain the result on the same drawing;
- whenever an operation is easier to see as a network, draw the neurons, arrows, residual paths, or attention routes rather than replacing them with a paragraph;
- slide code is limited to a short PyTorch fragment, normally four to seven lines. The complete executable path lives in the companion notebook.
- text-free editorial illustrations may support a conceptual hook; all labels, equations, arrows, and technical diagrams remain native Typst vectors.

Use native Typst/Fletcher for technical diagrams. Keep the reasoning and reveals close to Nipun's notes, with readable type rather than a handwriting font.

- Teal: observed context, token embeddings, keys.
- Blue: model-produced scores, queries, probabilities.
- Orange: targets, values, retrieved information.
- Green: correct/retained paths and residual additions.
- Red: loss, invalid future positions, failure modes.
- Ink/cream: conventions, shapes, and unchanged infrastructure.
- Rows always mean tokens/queries; columns in an attention map always mean candidate source tokens/keys.
- The persistent numerical case in Lecture 2 is `a a b -> i` with two-dimensional vectors. It is deliberately small enough to calculate by hand.

Overall narrative

1. What can neural networks do with text?
2. Can generation be reduced to repeated classification?
3. Build the full fixed-window character model.
4. Characters are only one possible tokenization.
5. Generalize the same objective to natural-language tokens.
6. Calculate how concatenation grows with context length.
7. Try averaging and test what information it loses.
8. Make the weights depend on the query.
9. Name the three roles Query, Key, and Value.
10. Let every token retrieve: self-attention.
11. Prevent future leakage: causal masking.
12. Let several retrieval mechanisms operate in parallel: MHA.
13. Combine MHA with the MLP and residual ideas students already know.
14. Stack blocks and attach the same next-token objective.
15. Compare the two models on the same character prediction task.

Lecture 1: Next-token prediction

Timing: ~80 minutes. Actual deck: 51 conceptual frames. The base planning map below is frames 1–50, plus the inserted loss-code frame listed above.

Act 1 — Why language? (frames 1–12)

#	Teaching frame	Content, worked example, and visual direction
1	Deep learning for language	Almost empty opening: “How can a neural network work with language?” Reveal a native-vector collage of email→spam, review→sentiment, article→summary, English→Hindi, context+question→answer, prompt→continuation. No equations.
2	One input → one label	Review: “The acting was wonderful but the ending was disappointing.” Show a binary softmax with 0.61/0.39. Ask whether this is fundamentally different from image classification.
3	Intent classification	Three cards: book a Delhi flight→`BOOK_FLIGHT`; account balance→`CHECK_BALANCE`; cancel booking→`CANCEL_BOOKING`. Conclude `x -> y`.
4	Token-level prediction: named entities	“Sundar Pichai visited IIT Gandhinagar on Monday.” Reveal aligned PERSON/O/ORG/DATE tags. Establish sequence→aligned sequence.
5	Translation changes the sequence length	English sentence enters a model and exits as Hindi. Ask whether token counts must match. Establish sequence→different-length sequence.
6	Summarize a notice	Compress a six-line IITGN-style notice to one sentence. Leave the question “How does the model decide what matters?” unanswered; this phrase returns in attention.
7	Answer a question using the context	Context about IITGN’s Palaj campus, question “Where?”, answer “Palaj.” Seed the idea that useful context depends on the question.
8	Represent similar meanings with nearby vectors	Put “student solved assignment” close to “learner completed homework” and far from a rain sentence in a small 2-D vector map.
9	Generation has many valid answers	Prompt: “The student opened the laptop and …” Collect several plausible continuations. Ask whether there is one correct answer.
10	Generation is still classification	Reveal a softmax over plausible next tokens. Link directly to categorical classification.
11	Generate a name	“Can we generate Indian names?” Reveal a list from `aabid` to `zeel`.
12	What exactly should we predict?	Character, word, or something between? Name the general unit “token,” then choose one character for the first complete model. Defer BPE.

Act 2: Build a character model (frames 13-34)

#	Teaching frame	Content, worked example, and visual direction
13	The task: generate an Indian name	Generate Indian names; vocabulary `a…z` plus one boundary token `-`; `|V|=27`. Clarify that the toy `-` plays BOS/padding/EOS roles only for this derivation.
14	Generation is repeated 27-class classification	`a a b -> ?`; reveal a 27-class probability strip with `i` high. State `f_theta(a,a,b)=p(c_next|a,a,b)`.
15	Choose a context length	Commit to `k=3`; define `p(x_t|x_{t-3:t-1})`. Do not criticize it yet.
16	Construct the training examples	Progressive reveal of the six exact context-target rows.
17	Slide the context window	Animate a width-three sliding window across `---aabid-`. Add a "remember this" bookmark for causal parallel training later.
18	Repeat for every name	A name of length `L` yields `L+1` targets including EOS; total examples are `sum_n(L_n+1)`.
19	How do we turn characters into numbers?	Put `a b i` beside the MLP's requirement `x in R^d`. Invite one-hot as the obvious first answer.
20	One-hot encoding	Show three one-hot rows and equal zero dot products. Ask what similarity they encode.
21	Learn an embedding for each character	Introduce `E in R^(27×d)` and choose `d=2`; highlight one row per character.
22	Look up the embedding	Map `a,b,i`→IDs→explicit rows, using the same colors from token through table to vector.
23	Check the shapes	`X` has shape `(3,)`, `E[X]` has shape `3×2`; write the three-row matrix. Use the same shape checks when Q/K/V appears.
24	Concatenate the embeddings	Flatten the three rows into one six-dimensional vector. Keep position-colored segments visible.
25	Why concatenate?	Compare `a,b,i` with `i,b,a`; concatenation preserves slots while a plain sum does not. Plant the need for a more scalable representation of position.
26	Pass the vector through an MLP	Show `h1=sigma(W1 h+b1)` and exact dimensions `W1 in R^(H×6)`.
27	Why exactly 27 outputs?	One output logit per possible next character; conclude this is 27-class classification.
28	Turn the 27 scores into probabilities	Give a few toy logits, normalize them, and draw a probability bar chart with target `i`.
29	Compute the loss	Compare `-log .8≈.223` with `-log .01≈4.605`. The correct character should receive probability mass.
30	Learn the embeddings and MLP weights	Box the embedding table and both MLP layers. Emphasize that backprop learns the representation and classifier jointly.
31	When could two embeddings become similar?	Similar context produces related gradient pressure; show an explicitly illustrative 2-D character embedding, without claiming human-like character semantics.
32	Sample, append, repeat	Start from `---`, sample `a`, shift to `--a`, and continue until EOS to generate `aabid`.
33	Training and generation	Two-column comparison: known target/loss/update/many windows versus unknown target/choose/append/autoregressive loop.
34	Our character language model	Large equation `p(x_t|x_{t-3:t-1})`. It is tiny and character-level, but it predicts the next token from previous tokens.

Act 3 — What should a token be? (frames 35–44)

#	Teaching frame	Content, worked example, and visual direction
35	Why begin with characters?	They made the first model inspectable; show how long “deep learning is amazing” becomes character by character.
36	Character tokens	`transformer -> t|r|…`; tiny vocabulary, long sequence, almost no unknown strings.
37	Word tokens	Natural short sequence, but `walk/walks/walked/walking/walker/walkability` expands vocabulary.
38	What happens to an unseen word?	A test string such as `hyperhappiness` collapses to `<UNK>` under a word vocabulary.
39	Can we split text at spaces?	Use “IITGN में आज transformer lecture है :)”, plus a URL and code fragment. Ask what a word is across scripts, punctuation, emoji, and code.
40	Subword tokens	Split `unbelievable`, `tokenization`, and a rare scientific word into reusable chunks. Keep BPE mechanics for a later lecture.
41	Choose the token size	Vector diagram from tiny-V/long-T characters through moderate subwords to huge-V/short-T words.
42	Compare the trade-off	Table: vocabulary size, sequence length, OOV behavior, semantic convenience. Highlight that attention cost depends strongly on `T`.
43	Tokens become integer IDs	Small vocabulary table for “The cat sat on the mat” and the corresponding ID sequence.
44	Use token IDs in the model	Define `x_1,…,x_T`, `x_i in {1,…,V}`. Display whole words for readability while treating them mathematically as tokens.

Act 4: Predict natural-language tokens (frames 45-50)

#	Teaching frame	Content, worked example, and visual direction
45	Change the tokens, keep the model	Place `a a b -> i` beside `cat sat on -> the`; identical embedding→concatenate→MLP→softmax pipeline.
46	What could come next?	“The cat sat on the ___.” Gather mat/floor/chair/sofa, then show a distribution instead of one truth.
47	Construct examples from a sentence	`deep -> learning`, `deep learning -> is`, … . Relate prefixes to the earlier sliding windows.
48	Multiply the next-token probabilities	Work `p(deep learning is fun)` into the chain-rule product, one factor at a time.
49	How should we represent the context?	`L(theta)=-sum_t log p_theta(x_{t+1}|x_{<=t})`. The objective is settled; context representation is the open problem.
50	What if the context keeps growing?	`aab -> i` beside `The cat sat on the -> mat`; ask “How should we represent arbitrarily long context?”

Lecture 2: How attention works

Timing: ~80 minutes. Actual deck: 59 conceptual frames. The base planning map below is frames 51–104, plus the five Lecture 2 inserts listed above. The formula `softmax(QK^T/sqrt(d_k))V` must not appear until the single-query mechanism is understood numerically.

Act 5: How should we combine the context? (frames 51-63)

#	Teaching frame	Content, worked example, and visual direction
51	The fixed-window pipeline	Restore the familiar token→embedding→concatenate→MLP diagram.
52	What if the window is five tokens?	Concatenation still works. Let students say so.
53	What if the window is 100 tokens?	Input is `100d`; first-layer parameter count grows with context.
54	What if the window is 10,000 tokens?	A comically long native-vector concatenation strip makes the scaling failure visible.
55	The window is part of the architecture	Different `k` means a different input dimension and different first layer.
56	Which context words help?	Coffee/cup pair: which earlier noun matters changes with the continuation. Define selective context.
57	Useful evidence can be far away	"The keys to the old wooden cabinet … are missing." Draw the long dependency from `keys` to `are`.
58	Who does `she` refer to?	Alice/Priya/`she` example. Attention can expose evidence; it does not guarantee an unambiguous answer.
59	A lookup embedding cannot see context	Contrast river bank and financial bank; the lookup row starts the same.
60	What do we want from a context summary?	From `v_1,…,v_n`, construct one fixed-size `h` that emphasizes useful information.
61	First try: average the token vectors	`h=(1/n)sum_j v_j`; check that the output size stays fixed as the context grows.
62	The mean gives every token the same weight	Function words and content words receive `1/n`. Equal weight cannot be task-dependent.
63	The mean also forgets order	`mean(dog,bites,man)=mean(man,bites,dog)`. Averaging is permutation-invariant. Foreshadow the position signal added before Q/K/V later in the same lecture.

Act 6 — From weighted averages to Q/K/V (frames 64–74)

#	Teaching frame	Content, worked example, and visual direction
64	Next try: use a weighted average	`h=sum_j alpha_j v_j`, `alpha_j>=0`, `sum alpha_j=1`.
65	Read the weights as mixing amounts	A token strip with 0.05/0.65/0.10/0.20 shows where the output came from.
66	Who should choose the weights?	Commit question before the reveal: position, token identity, or current information need?
67	One fixed set of weights is not enough	The best source token changes for sentiment, pronoun resolution, and next-token prediction.
68	Compute the weights from a query	The current position emits a description of what it seeks; candidates advertise what they match.
69	Attention is a soft lookup	Align the lookup vocabulary with `q`, `k_j`, `q^T k_j`, `v_j`, and the weighted return; define `j` as the source number here. Beside it, a text-free card-catalog illustration shows several sources contributing by different amounts to one return.
70	Query: what am I looking for?	Define `q`; examples: “which noun controls this verb?” or “what entity does this pronoun refer to?”
71	Key: when should this source match?	First show source 1 with `k_1`, source 2 with `k_2`, and source 3 with `k_3`. Then generalize: the small `j` is the source number, and matching uses `q^T k_j`.
72	Value: what should this source send?	Each source token has `v_j`; matching features and payload need not be identical.
73	Compute attention for one query	Bridge the indices explicitly: for current token `i`, write `q=q_i` and compare it with sources `j=1,...,T`. Score `s_j=q^T k_j`; normalize `alpha=softmax(s)`; retrieve `h=sum alpha_j v_j`.
74	Write the three steps in one line	Box `Attention(q,K,V)=softmax(qK^T)V`. State row/column convention.

Act 7 — Work every number for `a a b -> i` (frames 75–84)

#	Teaching frame	Content, worked example, and visual direction
75	Attention on the character example	The MLP concatenates `e_a,e_a,e_b`; attention uses the last token’s query to combine the visible values.
76	Pick tiny vectors we can check by hand	Use `q_b=[0,1]`; keys `k_a=[1,0]`, `k_a=[1,0]`, `k_b=[0,1]`; values `v_a=[2,0]`, `v_a=[2,0]`, `v_b=[0,2]`. Say explicitly that score scaling is postponed until frame 102 so the first arithmetic stays inspectable.
77	Step 1: compare the query with each key	Dot products give scores `[0,0,1]`. Label every multiplication, not just the result.
78	Step 2: turn scores into weights	`softmax([0,0,1])≈[.212,.212,.576]`; draw aligned bars.
79	Step 3: mix the values	`h=.212[2,0]+.212[2,0]+.576[0,2]=[.848,1.152]`.
80	The context vector goes back to the language model	Feed `h` through the familiar MLP/LM head; show a distribution with `i` high. Attention changed the context vector, not the objective.
81	What if we change the query?	Change the query to `[1,0]` and recalculate the weights, keeping the keys and values fixed.
82	Why a soft lookup?	Several sources can contribute, every path remains differentiable, and ambiguity can be represented.
83	Which parts are learned?	Let `h_i` denote the current vector state for token ID `x_i`; introduce `q_i=h_iW_Q`, `k_i=h_iW_K`, `v_i=h_iW_V`.
84	Shapes for one query	`q in R^(d_k)`, `K in R^(T×d_k)`, `V in R^(T×d_v)`, output in `R^(d_v)`.

Act 8 — Every token retrieves: self-attention (frames 85–94)

#	Teaching frame	Content, worked example, and visual direction
85	Why should only the final token ask?	Let every position issue its own query against the same sequence.
86	Let every token make Q, K, and V	`Q=XW_Q`, `K=XW_K`, `V=XW_V`; show exact `T×d` shapes.
87	Compare every query with every key	`S=QK^T` with `S_ij=q_i^T k_j`; rows look, columns are looked at.
88	Softmax each query row	Apply softmax across keys, never across query rows. Add a row-sum check.
89	Mix values for every query	`A=softmax(S)`, `O=AV`; verify `T×T` times `T×d_v` gives `T×d_v`.
90	The full self-attention formula	Only now show `Attention(Q,K,V)=softmax(QK^T/sqrt(d_k))V`.
91	Each output vector depends on its context	Two identical `a` lookup embeddings can produce different output states because their visible contexts/positions differ.
92	Attention alone does not know order	Permuting rows permutes outputs when there is no position signal or fixed mask; the operation alone has no absolute or relative order signal.
93	Add position before making Q, K, and V	`h_i^(0)=E[x_i]+p_i`; visualize token content plus position as two colored components.
94	Two simple ways to add position	One compact comparison; both inject order, with different extrapolation/parameter trade-offs. Keep advanced variants optional.

Act 9 — Causality and parallel next-token training (frames 95–104)

#	Teaching frame	Content, worked example, and visual direction
95	Can a training token peek at the answer?	Commit question. If `a` can see the later `b`, predicting `b` is cheating.
96	Without a mask, future tokens leak in	Show an unmasked all-to-all attention matrix over `---aabid-`; future tokens directly reveal targets.
97	Draw the causal mask	Allowed when `j<=i`, forbidden when `j>i`; triangular matrix with red future cells.
98	Mask future scores before softmax	Add `M` before softmax; work one row to show forbidden probabilities become exactly zero.
99	Read a masked attention matrix	Row 1 sees itself; row 2 sees 1–2; row `t` sees `1…t`. Each row still sums to one.
100	One forward pass predicts every next token	Position `t` predicts `x_{t+1}`. Align input tokens, hidden states, and shifted targets.
101	Predict all six targets in parallel	The six `aabid` targets that the MLP treated as separate examples now arise in one masked pass. Emphasize: same targets, but each attention row may use its entire allowed prefix rather than exactly three slots.
102	Why scale the dot product?	Variance of a dot product grows with `d_k`; large logits saturate softmax. Scale to keep scores in a trainable range.
103	Check the attention calculation	Symptom→suspect→test strip: row sums not one→wrong softmax axis; future mass→mask error; identical outputs→missing positions or degenerate projections.
104	What we built today	Average→weighted average→query-dependent weights→Q/K/V→self-attention→causal mask. Ask what several attention heads could add.

Lecture 3: Build a Transformer

Timing: ~80 minutes. Actual deck: 54 conceptual frames. The base planning map below is frames 105–154, plus the four Lecture 3 inserts listed above.

Act 10 — Multi-head attention (frames 105–116)

#	Teaching frame	Content, worked example, and visual direction
105	Recall one attention head	One query space, one key space, one value space, one weighted retrieval per token.
106	Why use more than one head?	A token may simultaneously need local syntax, entity identity, and long-range topic information.
107	Give each token several attention heads	Three color-coded heads: “nearest modifier?”, “which entity?”, “what topic?” Avoid claiming these are guaranteed learned semantics.
108	Each head learns its own projections	`Q_h=XW_Q^h`, `K_h=XW_K^h`, `V_h=XW_V^h`.
109	One head may look nearby	Local/previous-token weights on a short sentence. Keep the same row/column convention.
110	Another head may look back to a person	Long-range noun/pronoun or subject/verb relation on the same sentence.
111	A third head may notice a boundary	Copy/entity or delimiter behavior on the same tokens. Explicitly label maps as pedagogical possibilities.
112	The heads do the same calculation separately	`head_h=Attention(Q_h,K_h,V_h)` with shapes `T×d_v`.
113	Put the head outputs side by side	`Concat(head_1,…,head_H)` restores a width of `H d_v`.
114	Mix the concatenated features with a linear layer	Output projection returns to model width `d_model`.
115	Check the widths before moving on	Work the common case `d_k=d_v=d_model/H`; show why concatenation has width `d_model`.
116	What can an attention map tell us?	Heads are learned distributed computations; visual patterns can be inspected but should not be over-narrated.

Act 11 — Build one Transformer block from familiar parts (frames 117–129)

#	Teaching frame	Content, worked example, and visual direction
117	Attention mixes rows; the FFN mixes features	Contrast information exchange between token rows with nonlinear feature recombination inside each row.
118	Give every token the same small neural network	Position-wise FFN: `FFN(x)=W_2 phi(W_1x+b_1)+b_2`. Same weights at every token.
119	Across tokens, then within a token	Two-axis grid visual distinguishes across-position mixing from within-position feature transformation.
120	Why does the FFN widen in the middle?	`d_model -> d_ff -> d_model`; intermediate width creates nonlinear feature combinations.
121	Bring back the ResNet shortcut	Ask what happens if a new sublayer is initially unhelpful. Residual paths preserve and refine representations.
122	Add the attention output to its input	`r=x+MHA(x)`; draw an uninterrupted residual stream with a branch through attention.
123	Add the FFN output to its input	`y=r+FFN(r)`; repeat the same visual grammar.
124	LayerNorm works across one token's features	Work `[1,3,5] -> [-2,0,2] -> [-1.22,0,1.22]`, then name the general per-row operation.
125	Where should LayerNorm go?	Show both equations; choose modern pre-norm for the remainder and mark the architectural convention explicitly.
126	Put the sublayers together	`x -> LN -> causal MHA -> + -> LN -> FFN -> +`. One centerpiece native-vector diagram.
127	Write the block as two updates	`u=x+MHA(LN(x))`; `y=u+FFN(LN(u))`. Match every equation color to the diagram.
128	Stack several blocks	Repetition deepens the sequence of retrieval and transformation; parameters differ by layer.
129	The next block uses the updated token states	One layer can route directly; several layers can retrieve, transform, and retrieve again. Avoid promising human-readable reasoning steps.

Act 12 — The complete decoder-only language model (frames 130–142)

#	Teaching frame	Content, worked example, and visual direction
130	Begin with token and position information	Token IDs→token embeddings + positional information; resulting matrix is `T×d_model`.
131	Turn the final token state into token scores	Final state at each position→linear projection to `V` logits→softmax.
132	The decoder-only model from end to end	Tokenizer→embedding+position→`L×` decoder blocks→LM head→next-token distributions.
133	Slide the targets one place to the left	Align `<BOS> deep learning is` with targets `deep learning is fun`; no handwaving about which position predicts what.
134	During training, we know the whole sentence	During training, each row receives the true prefix under the mask; no sampled token is fed within that pass.
135	Score every next-token guess	`L=-sum_t log p(x_{t+1}|x_{<=t})`; optional padding mask excludes non-data positions.
136	The next-token objective stays the same	Place the `aabid` cross-entropy and the Transformer token loss in one equation frame.
137	Training: one pass, many next-token guesses	Whole sequences, parallel masked positions, loss, backprop, parameter update.
138	Generation: one new token, then go again	One current prefix, final-position distribution, choose/sample, append, repeat.
139	Generation, first step	Start `<BOS> The`; show attention flow and choose `student`.
140	Generation, one step later	Append and recompute/extend; choose `opened`. Keep probabilities visible.
141	Cache the keys and values from earlier positions	Past K/V rows do not change during autoregressive decoding; reuse them instead of recomputing every old projection.
142	What does a longer context cost?	Vanilla attention stores an `T×T` training map and decoding KV cache grows with `T`; direct access has a real memory/compute cost.

Act 13: Compare the models (frames 143-154)

#	Teaching frame	Content, worked example, and visual direction
143	Three common ways to arrange Transformer blocks	Encoder-only, decoder-only, encoder–decoder cards; distinguish visibility and typical objective.
144	Encoder-only: read the whole input	Bidirectional self-attention for contextual representation/classification; BERT as an example, not a decoder.
145	Decoder-only: read only the prefix	Causal self-attention for next-token generation; GPT-style model is the deck's main construction.
146	Encoder-decoder: read one sequence, write another	Source encoded bidirectionally, target decoded causally; useful for translation and conditioned generation.
147	Self-attention and cross-attention use the same calculation	Self: Q/K/V from same stream. Cross: queries from decoder, keys/values from encoded source.
148	The 2017 Transformer had an encoder and a decoder	One historical slide: encoder–decoder architecture from "Attention Is All You Need"; then return to the modern decoder-only focus.
149	Put the sequence models side by side	Table: fixed-window MLP, RNN, temporal convolution, self-attention—context flexibility, path length, training parallelism, and cost.
150	Longer memory has a price	Vanilla self-attention is `O(T^2)` in pair scores; RNN is sequential; fixed MLP hard-codes `k`. No architecture wins every axis.
151	What does attention give us?	Gives learned content-dependent routing. Does not guarantee factuality, reasoning, interpretability, or unlimited memory.
152	Four quick checks	Four claims: "weights are explanations," "MHA means one linguistic rule per head," "masking is only used at generation," "a Transformer removes the MLP." Students correct each.
153	Compare our two language models	Side-by-side pipelines: fixed three-row concatenation and causal self-attention over an arbitrary prefix. Highlight unchanged tokenizer IDs→embeddings→logits→CE→sampling skeleton.
154	Can you finish this sentence?	Students complete: "A Transformer language model differs from our `aabid` MLP mainly because ..." Final answer: it builds each context representation by repeated, position-aware, causally masked, learned retrieval plus per-token MLP transformations.

Short-on-time routes

- Lecture 1: keep 1–2, 4–7, 9–17, 21–34, 40–50; move 3, 8, 18–20, 35–39, and 41–44 to handout review.
- Lecture 2: keep 51, 53–57, 60–65, 68–81, 83–90, 92–93, 95–104.
- Lecture 3: keep 105–108, 112–115, 117–128, 130–142, 145, 147, 149–154.

Checks before teaching

1. The six `aabid` examples are consistent everywhere.
2. Every attention matrix states its row/column convention.
3. Every softmax identifies its normalization axis.
4. All Q/K/V and MHA shape equations compile and agree.
5. Future attention is zero after the causal softmax.
6. Training targets are shifted exactly one token.
7. Both handout and progressive-presentation PDFs compile.
8. All pages pass a visual overflow/contact-sheet inspection.
9. Technical diagrams remain native vector Typst/SVG. Generated illustrations support examples, with labels and equations added in Typst.
10. The companion PyTorch notebook executes from top to bottom and reproduces the tiny attention calculation used in the slides.

Companion material

- Minimal slide code appears only at the first embedding lookup, loss, single-query attention, full self-attention, causal mask, decoder block, shifted loss, and generation loop.

- The complete executable companion is `lecture11/notebooks/from_characters_to_transformers.ipynb`.

Primary references

- Nipun Batra, *Next Token Prediction* (the `aabid` teaching anchor): https://nipunbatra.github.io/ml-teaching/neural-networks/slides/next-token-prediction.pdf
- MIT 6.S191, *Deep Sequence Modeling*: https://introtodeeplearning.com/slides/6S191_MIT_DeepLearning_L2.pdf
- Stanford CME 295, Lecture 1, *Transformer*: https://www.youtube.com/watch?v=Ub3GoFaUcds
- Vaswani et al., *Attention Is All You Need*: https://arxiv.org/abs/1706.03762